Project Documentation

FARM INSIGHT

Course Name: Software Project Management

Professor: Rexhep Rada

APRIL 24, 2026

Leandro Bakollari – Software Engineering
Oresti Duka – Software Engineering

Abstract

This report presents the analysis, design, and final refinement of a Farm Management System developed to support the core operational needs of a farm in a simple and structured way. The system is designed around key functional areas including farm structure management, plant and harvest tracking, worker task management, finance and sales recording, inventory control, and reporting. The report consolidates all coursework stages into one coherent document by bringing together the requirements analysis, process and behavioral modelling, database and domain structure, architectural and structural design, and the final maintenance and evolution planning. The final outcome is a corrected and internally consistent system design that reflects the actual project scope, aligns the diagrams and data model with the supported user roles of manager and worker, and provides a practical foundation for future development.

1. Introduction

1.1 Project Overview

The Farm Management System is a simple software solution designed to help a farm manage its daily operations in one place. It supports the management of farm areas, plants, worker tasks, harvest and resource records, finance and sales activity, inventory, and basic reporting so that managers and workers can keep information organized and easier to track.

1.2 Scope of the System

The system supports the main operational needs of the farm, including:

- farm structure management
- plant management
- task and work tracking
- harvest and resource tracking
- finance and sales records
- inventory management
- reporting

1.3 User Roles

- Manager
- Worker

2. Requirements and Analysis Summary

2.1 Problem Statement

The farm currently lacks a simple integrated system for managing growing areas, plants, worker tasks, harvest records, finances, and inventory in one place. As a result, information can become scattered, harder to update, and more difficult to monitor, making daily farm operations less efficient and reducing visibility over work progress and farm activity.

2.2 Objectives

1. Manage farm and growing area records.
2. Manage plants and harvests.
3. Manage tasks for workers.
4. Record finance and inventory activity.
5. Generate useful summaries and reports.

2.3 Functional Scope Summary

The system will support the core day-to-day activities of the farm. It will allow managers to maintain farm, plot, and greenhouse records, manage plant information and harvest history, create and assign tasks to workers, track task progress, record expenses and sales activity, manage inventory items and movements, and view simple summaries and reports for operational and financial monitoring.

2.4 Prioritized Requirements / User Stories Summary

The system requirements were prioritized using the MoSCoW method to separate essential features from less critical ones. The highest priority is given to the core workflows needed for

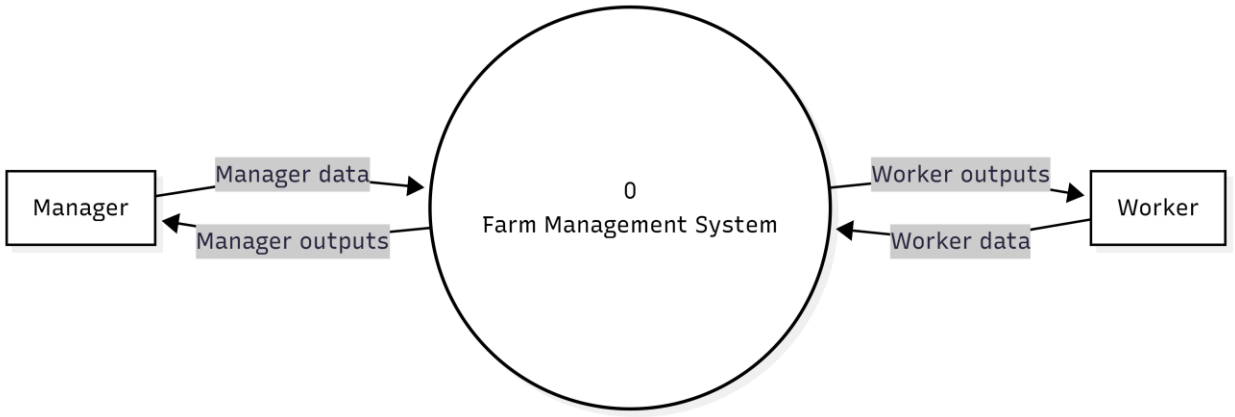
daily farm operations, while secondary features improve usability and record-keeping. More advanced analytics and automation are intentionally deferred to later releases.

PRIORITY	SUMMARY
MUST HAVE	User login, plant management, task management, finance and inventory recording
SHOULD HAVE	Task comments/history, recurring records, summaries
COULD HAVE	Deeper analytics, advanced filtering
WON'T HAVE THIS RELEASE	Unsupported external portals or advanced automation

2.5 Process and Data Modelling

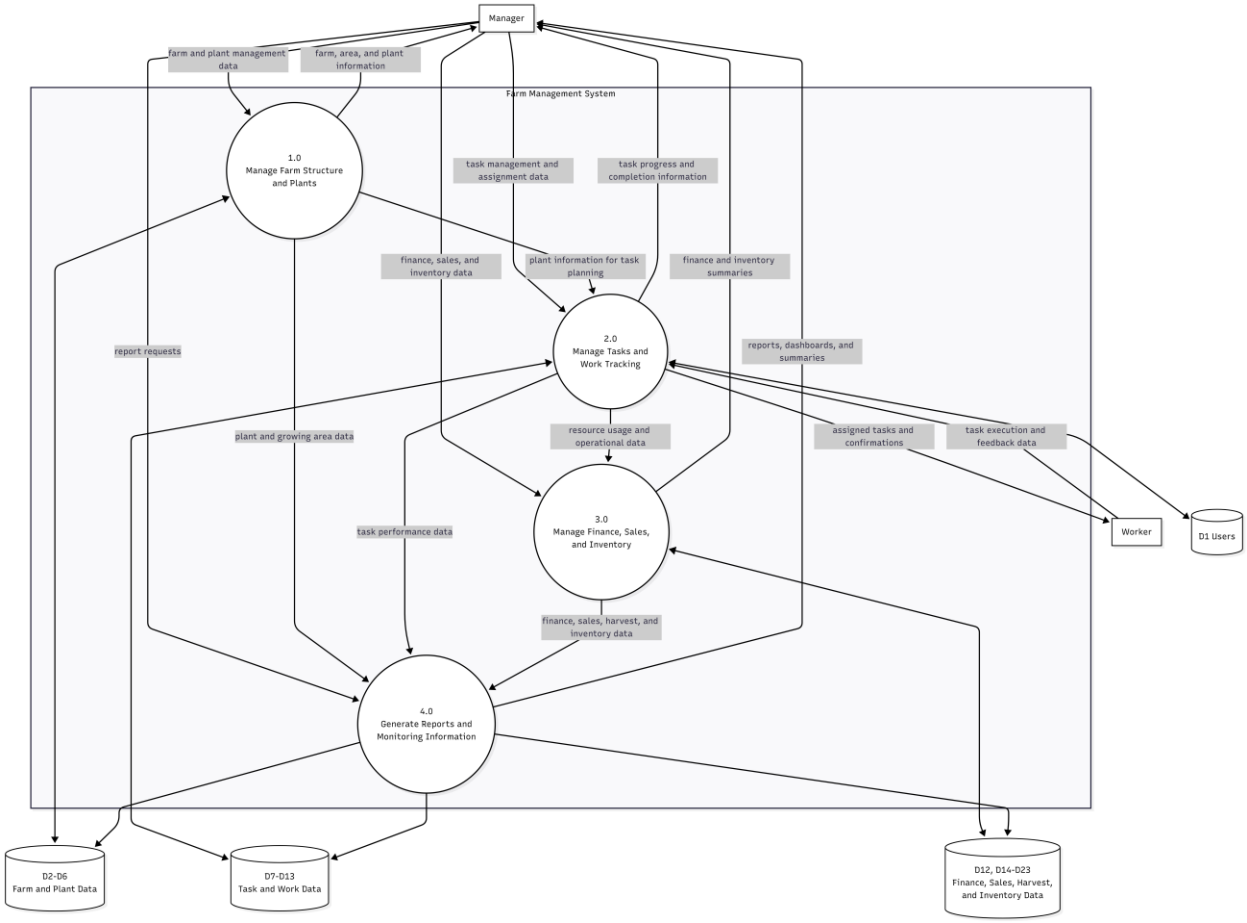
This section presents the main system models used to describe how the Farm Management System works. The diagrams show the overall system boundary, the main internal processes, and the interactions between users and system functions.

2.5.1 Context Diagram (Level 0 DFD)



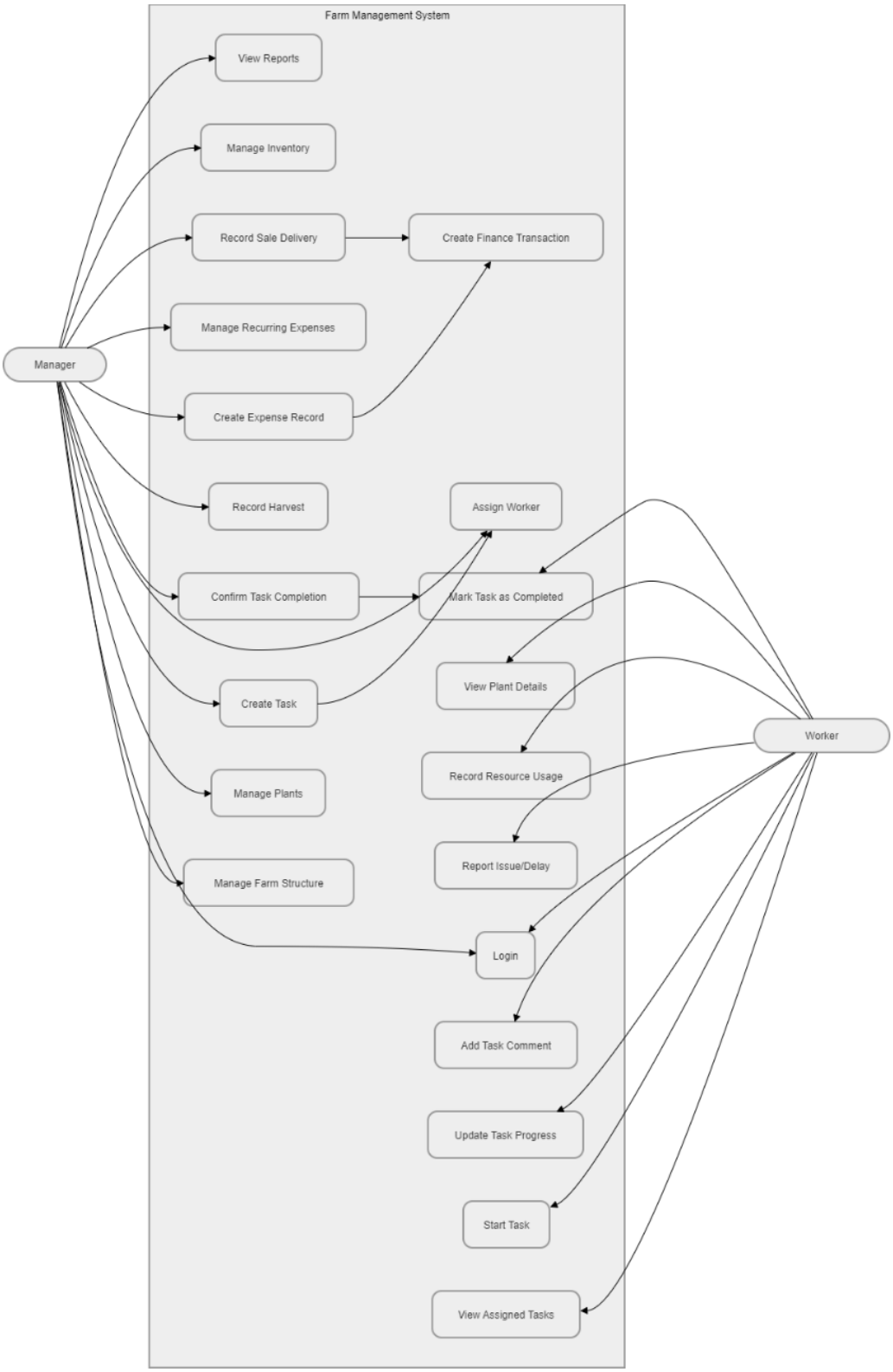
The Context Diagram shows the Farm Management System as one central process interacting with two external entities: Manager and Worker. The manager sends and receives farm, plant, task, finance, inventory, and report-related data, while the worker mainly exchanges task progress, comments, and resource usage information with the system.

2.5.2 Level 1 DFD



The Level 1 DFD breaks the system into four main processes: Manage Farm Structure and Plants, Manage Tasks and Work Tracking, Manage Finance, Sales, and Inventory, and Generate Reports and Monitoring Information. These processes use the main data stores defined in the requirements and database structure, including plants, tasks, harvest history, expense records, sales records, finance transactions, and inventory records.

2.5.3 Use Case Diagram



The Use Case Diagram shows how the Manager and Worker interact with the system's main functions. The manager handles administration and records such as farms, plants, tasks, expenses, sales, inventory, and reports, while the worker focuses on assigned tasks, progress updates, comments, resource usage, and task completion.

2.6 Key Use Case Narrative Summaries

The full use case narratives are available in the supporting requirements material, but the main report only includes short summaries of the most important flows. These summaries highlight the core actions and outcomes that are central to the system design.

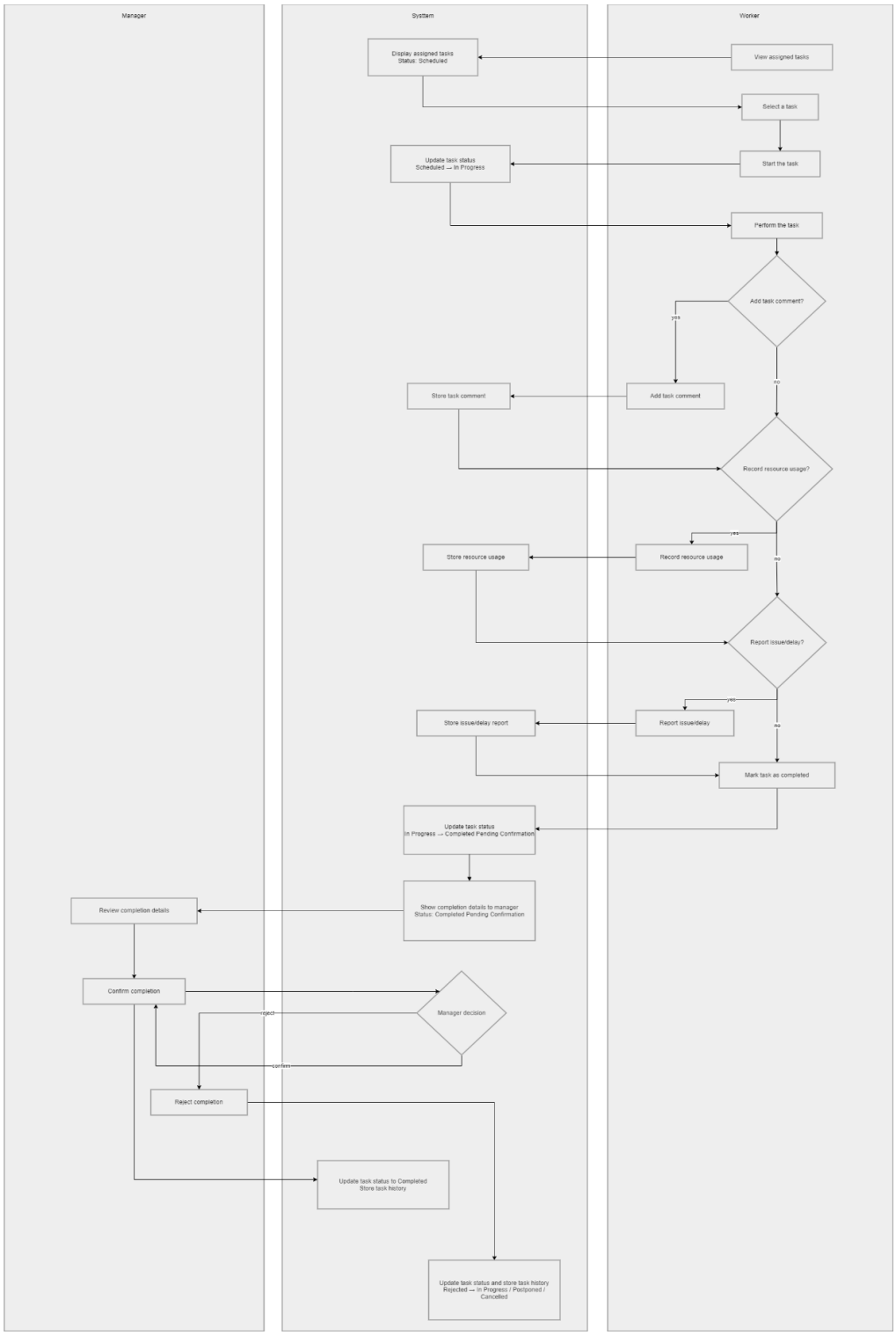
Use Case 1 – Create and Assign Task
Actor: Manager
Purpose: Create a task for a plant and assign one or more workers.
Main outcome: Task and assignment records are created, and the task becomes visible in the assigned workers' task lists.

Use Case 2 – Update Task Progress and Confirm Completion
Actor: Worker / Manager
Purpose: The worker updates progress and marks the task as finished, and the manager reviews and confirms completion.
Main outcome: Task status is updated and the progress and confirmation history are stored.

Use Case 3 – Create Expense Record
Actor: Manager
Purpose: Record a farm expense and its related financial entry.
Main outcome: The expense record is stored and a linked finance transaction is created for reporting and summaries.

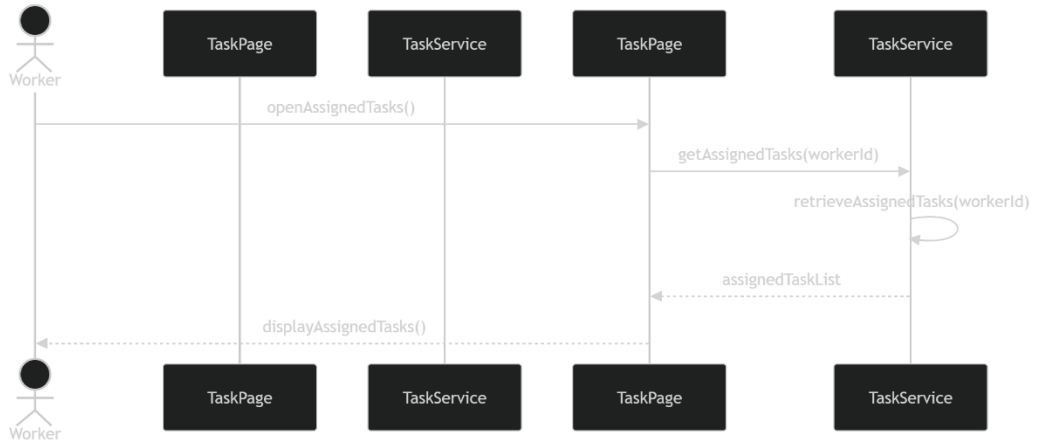
3. Behavioral Modelling

3.2 Activity Diagram with Swimlanes



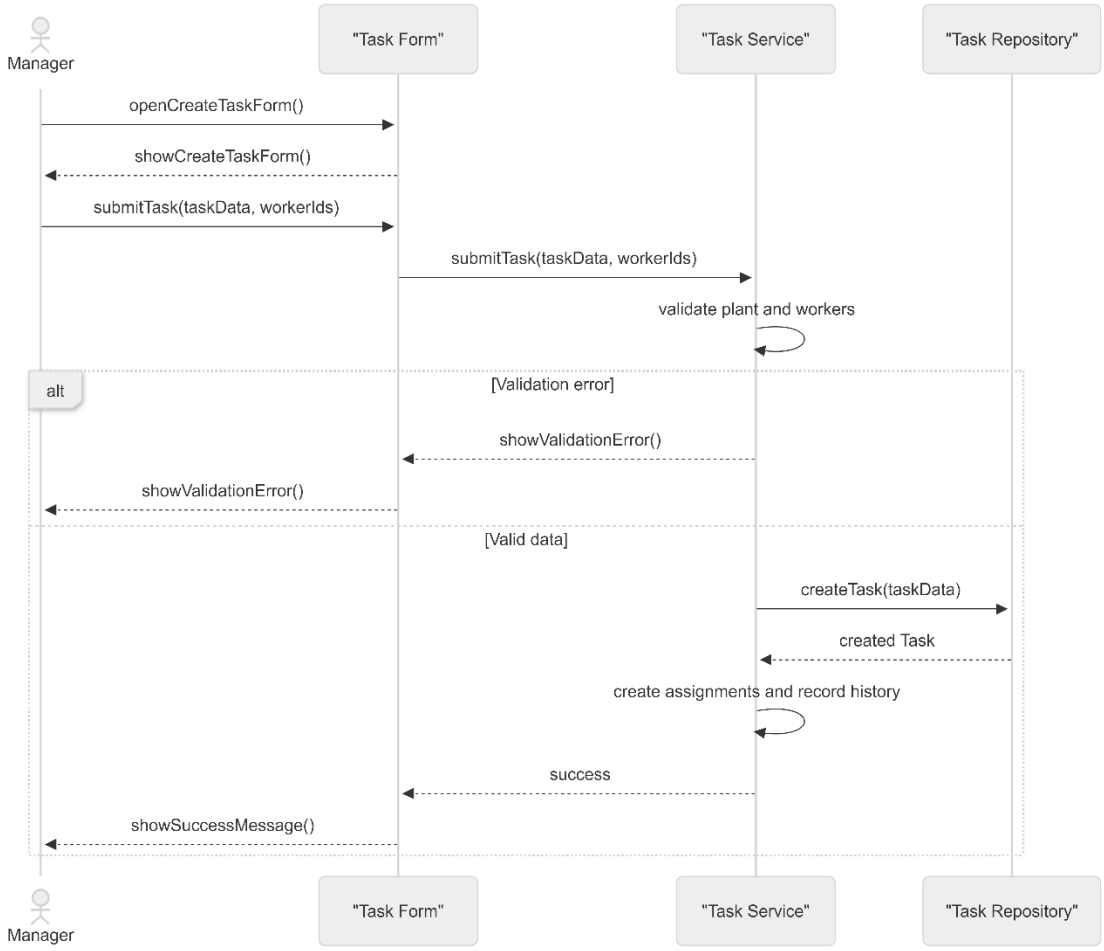
This activity diagram models the workflow Update Task Progress and Confirm Completion. It was chosen because it is a key multi-actor process involving the Worker, the System, and the Manager, and it reflects the real task lifecycle supported by the application. The flow shows task start, progress updates, optional comments or resource usage, completion marking, manager review, and final status update.

3.3 Sequence Diagram 1 – Simple



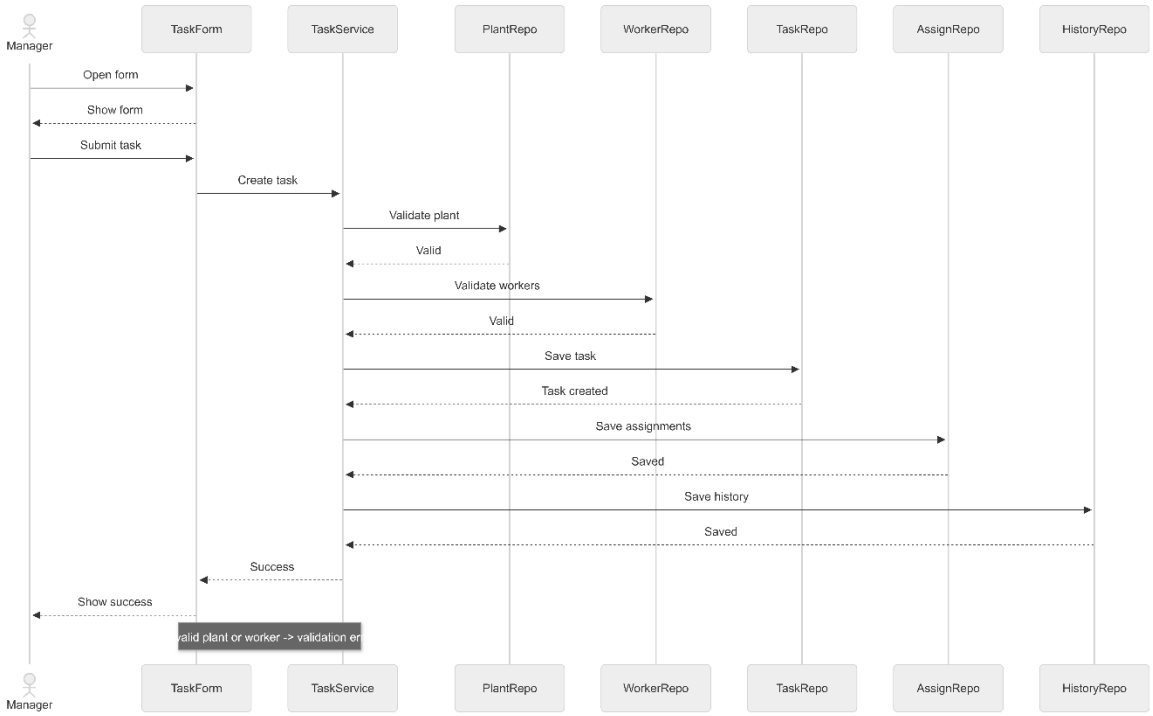
This sequence diagram models View Assigned Tasks. It is a simple interaction because it has a short flow, limited logic, and only a small number of participating objects, such as Worker, Task Page / Task Interface, and Task Service or Task Repository. The diagram shows the worker requesting assigned tasks and the system returning the relevant task list.

3.4 Sequence Diagram 2 – Complex



This sequence diagram models Create Task and Assign Worker. It is the complex sequence because it includes validation steps, multiple related objects, conditional behavior, and the creation of linked records such as the task, assignment records, and task history. It also fits the real system flow where a manager creates a task, the system validates plant and worker data, and then stores the related records.

3.5 Communication Diagram



This communication diagram is converted from Sequence Diagram 2 – Create Task and Assign Worker. It represents the same behavior as the complex sequence diagram, but instead of focusing on time order, it shows the structural links between the participating objects and the messages exchanged between them. The participants and message meaning remain the same so that both diagrams stay internally consistent.

3.6 Design Rationale

The behavioral models in this section were selected to show the most important workflows of the Farm Management System in a clear and balanced way. They focus on task-related interactions because task creation, progress, and completion are central to daily farm operations and closely linked with workers, plants, and system records.

A simple and a complex sequence diagram were included to show different levels of interaction. *View Assigned Tasks* represents a basic read-only process with few objects and little logic. *Create Task and Assign Worker* shows a more realistic workflow involving multiple objects, validation, related records, and repeated or conditional steps such as assigning more than one worker. Together, they provide a broader view of system behavior.

The activity diagram uses *Update Task Progress and Confirm Completion* because it suits swimlanes well. The workflow clearly shows responsibility moving between worker, system, and manager, making it more useful than a single-actor process.

Consistency of names across diagrams was also important. Using terms such as Manager, Worker, Plant, Task, Task Assignment, and Task History throughout keeps the models aligned with the system structure and database design, reducing confusion and improving overall clarity.

4. Database and Domain Structure Summary

4.1 Overview

The database is organized into major functional domains so that the system structure stays clear and consistent across the application. These domains group together the main data needed for users, farm areas, plants, tasks, finance, sales, inventory, and reporting, which also matches the modular system design used in the project.

4.2 Core Data Domains

Domain	Main tables
Users and Roles	users
Farm Structure	farms, plots, greenhouses
Plant Management	plant_stages, plants, harvest_history, resource_usage
Task Management	recurring_task_series, tasks, task_assignments, task_comments, task_history
Finance and Sales	finance_partners, expense_categories, recurring_expenses, expense_records, sales_deals, sales_deliveries, finance_transactions
Inventory	inventory_categories, inventory_items, inventory_movements

This domain grouping reflects the real schema and helps connect the report back to the actual implementation of the system.

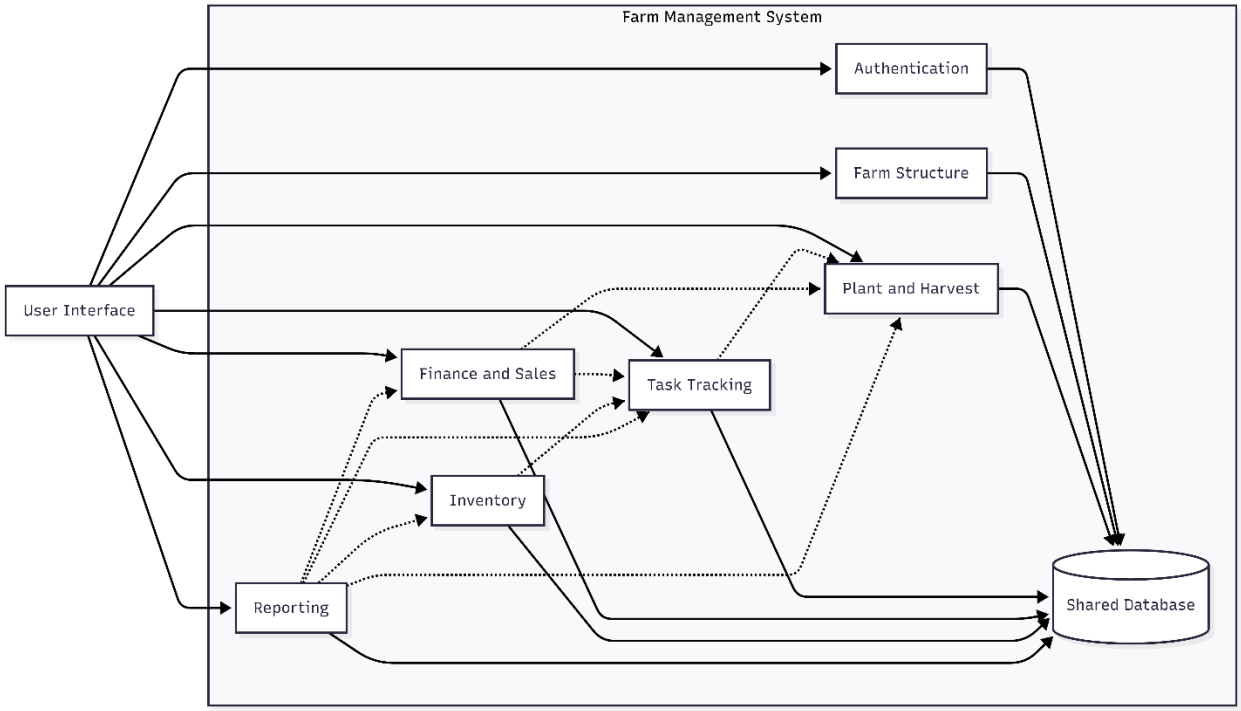
5. Architectural and Structural Design

5.1 Architectural Style Selection

The chosen architectural style for the Farm Management System is a layered modular monolith. This style is appropriate because the system is divided into clear modules such as user management, farm structure, plant management, task management, finance and sales, inventory, and reporting, while still sharing one application and one relational database. This makes the system easier to develop, test, and maintain for a small academic project.

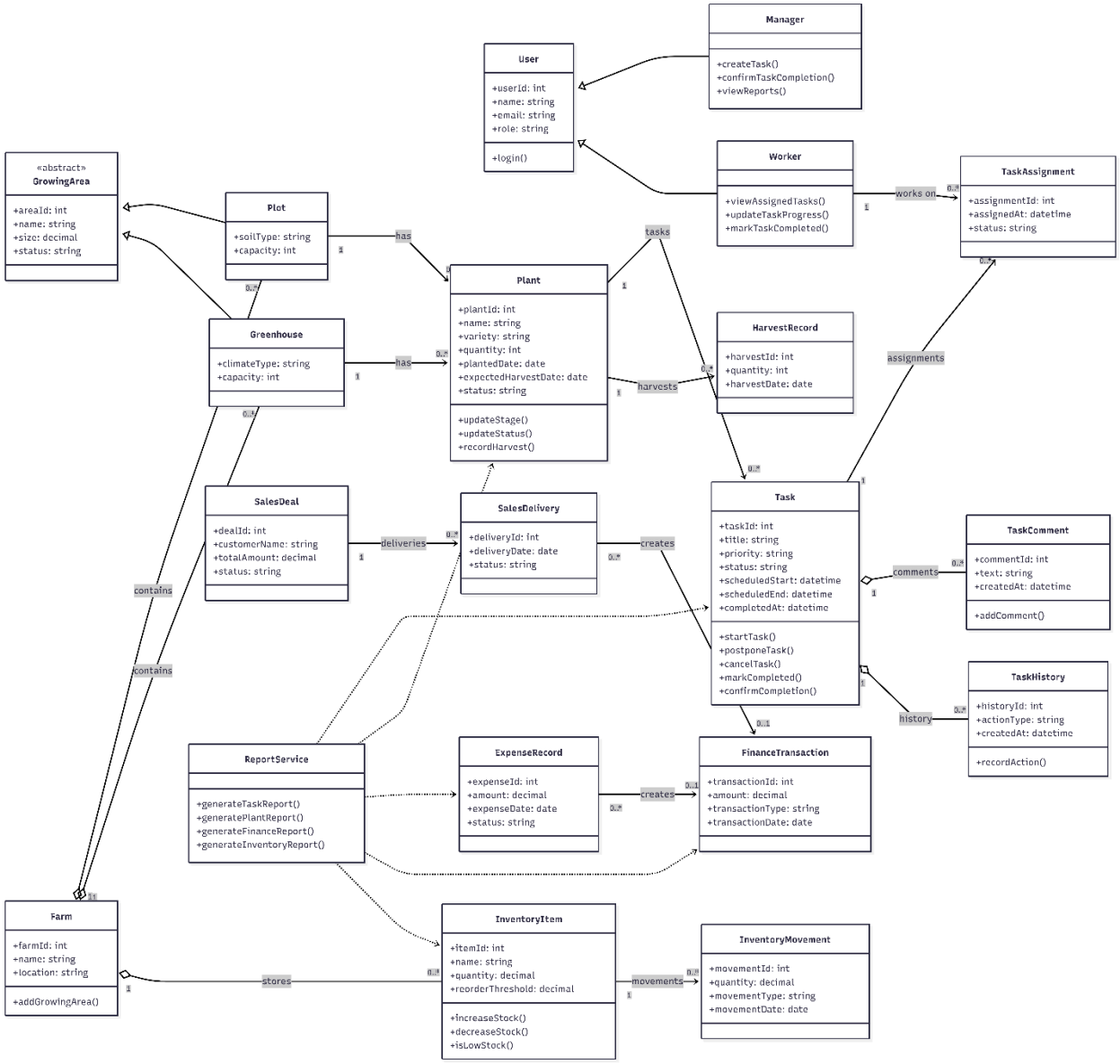
A rejected alternative was microservices architecture. Although it would separate subsystems into independent services, it would add unnecessary complexity in deployment, communication, security, and data consistency. Since the project uses one shared relational schema with many linked records across tasks, plants, harvests, finance, and inventory, a microservices approach would be too heavy for the size and purpose of this system.

5.2 Component Diagram



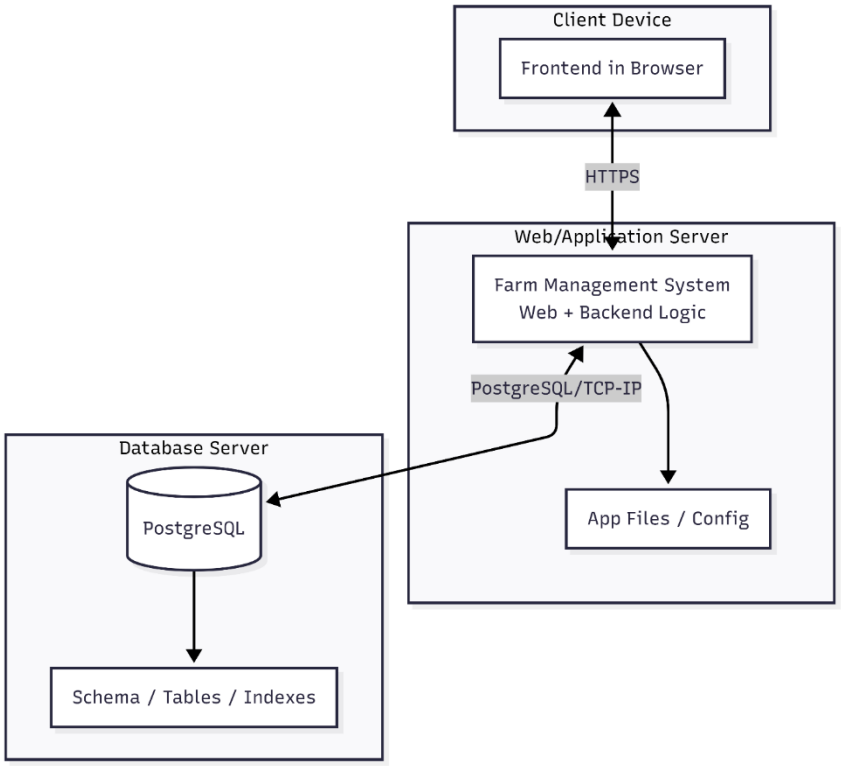
The Component Diagram shows the system as one integrated application divided into major subsystems: authentication and user management, farm structure management, plant and harvest management, task and work tracking, finance and sales management, inventory management, reporting and dashboard, and the shared database. It also shows that the main business components read from and write to the same database-backed structure.

5.3 Class Diagram



The Class Diagram represents the main object-oriented structure of the system using core entities such as User, Manager, Worker, Farm, GrowingArea, Plot, Greenhouse, Plant, Task, TaskAssignment, HarvestRecord, ExpenseRecord, SalesDeal, SalesDelivery, FinanceTransaction, and InventoryItem. It includes inheritance, associations, aggregation, dependencies, and multiplicities that are based directly on the database schema.

5.4 Deployment Diagram



The Deployment Diagram shows a simple web-based setup with three main nodes: Client Device, Web/Application Server, and Database Server. This reflects a realistic deployment for the project, where managers and workers access the system through a browser, the application server handles the business logic, and PostgreSQL stores the persistent data.

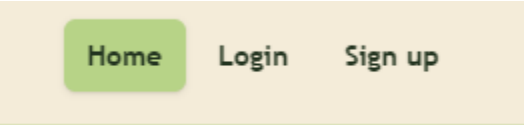
6. Interface Design Appendix Summary

This section presents the main interface structure of the Farm Insight system. The system is organized into separate pages based on user role and system functionality. The manager has access to the main management pages, including overview, finance, inventory, tasks, and plants. Workers have a separate page with limited access, focused mainly on viewing assigned work and relevant task information.

6.1 Navigation Overview

The interface is designed to keep the system simple and role-based. General users first access the Home page and then proceed to the Log In page. After logging in, the system redirects users depending on their role. Managers can access the main management dashboard and related

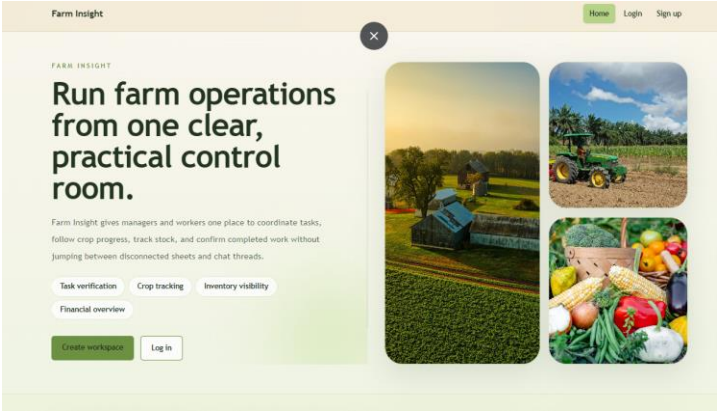
system sections, while workers are directed to a separate Workers page. This separation helps keep the interface clear and prevents workers from accessing management features such as finance, inventory, overview reports, and plant management.



6.2 General Pages

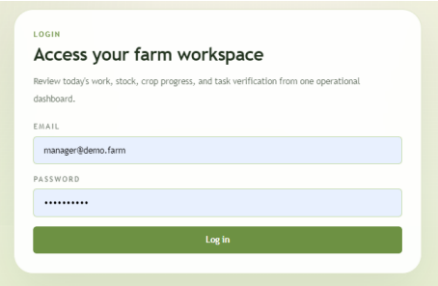
6.2.1 Home Page

The Home page is the first screen users see when entering the system. It introduces the Farm Insight system and provides access to the login option. This page acts as the entry point of the application.



6.2.2 Log In Page

The Log In page allows users to enter their credentials and access the system. After authentication, the system checks the user role and redirects them to the correct area. Managers are sent to the management section, while workers are sent to the Workers page.

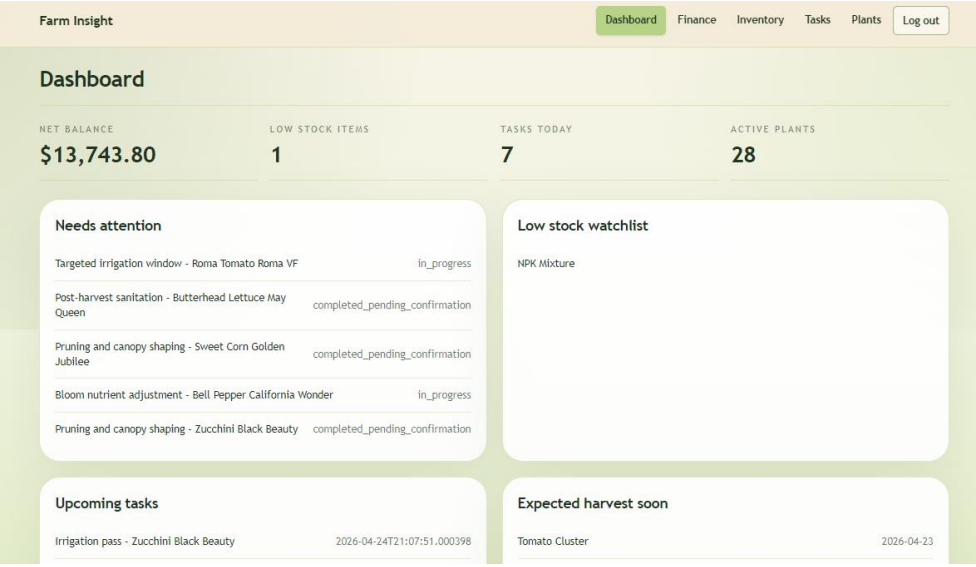


6.3 Manager Interface Pages

The manager interface contains the main control pages of the system. These pages are only accessible to users with manager permissions. They allow the manager to monitor farm activity, manage resources, organize tasks, track finances, and view plant-related information.

6.3.1 Overview Page

The Overview page acts as the manager's main dashboard. It provides a general summary of the farm's current status, including important statistics, recent activity, upcoming tasks, and key operational information.

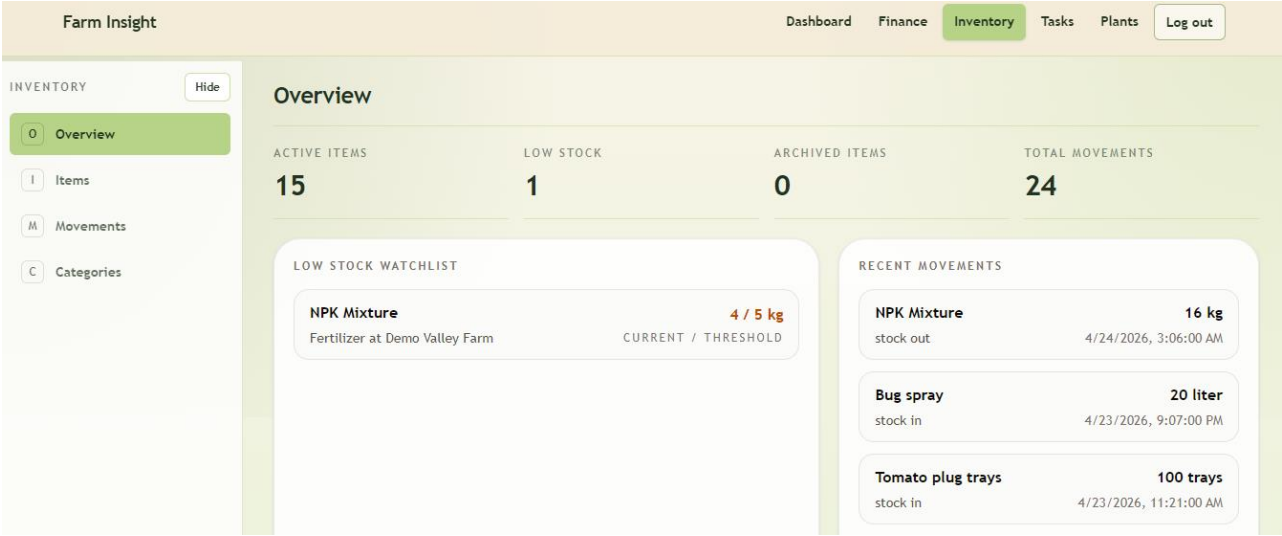


6.3.2 Finance Page

The Finance page allows the manager to view and manage financial records. This may include expenses, income, sales-related values, and overall financial summaries. The purpose of this page is to help the manager track the financial condition of the farm.

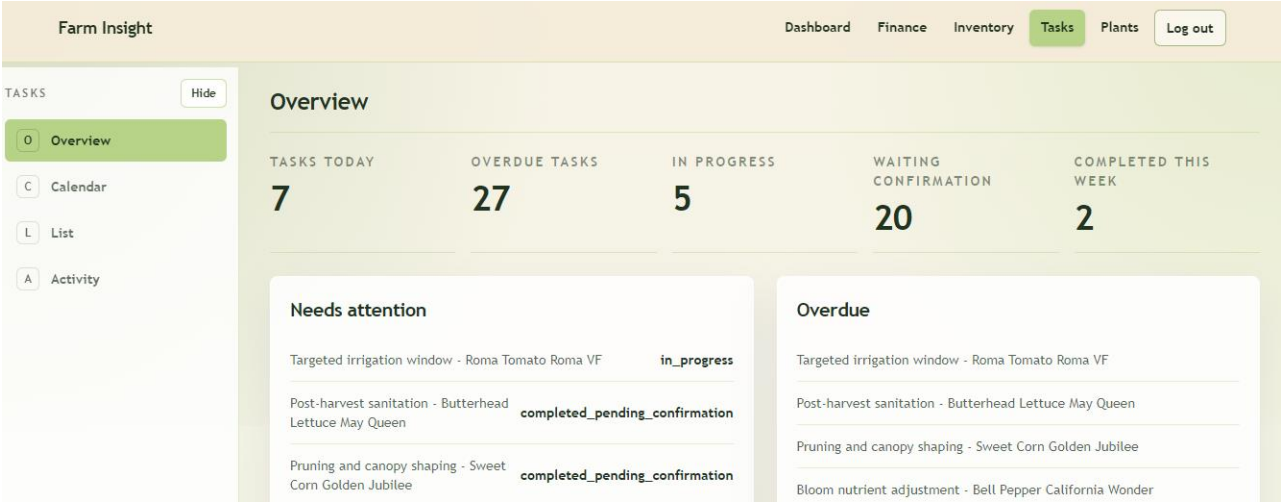
6.3.3 Inventory Page

The Inventory page is used to manage farm resources and stored items. This can include seeds, tools, fertilizers, equipment, harvested products, or other materials used in daily farm operations. The manager can use this page to monitor available stock and identify items that need to be updated or restocked.



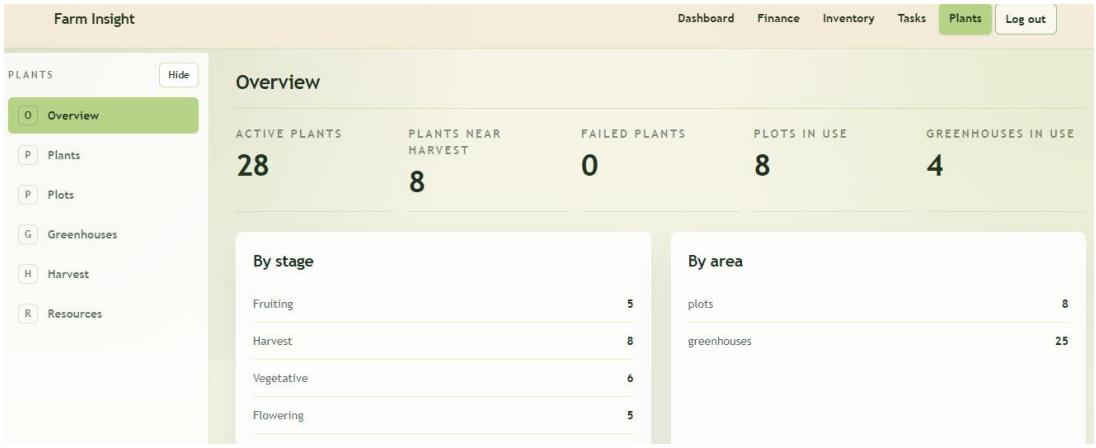
6.3.4 Task Management Page

The Task Management page allows the manager to create, assign, view, update, and organize farm tasks. Tasks can be connected to workers, dates, locations, and descriptions. This page supports the planning and monitoring of daily or weekly farm work.



6.3.5 Plant Management Page

The Plant Management page is used to manage information about plants and growing areas. It allows the manager to track crops, planting areas, growth stages, and related plant information. This page helps organize the agricultural side of the system.

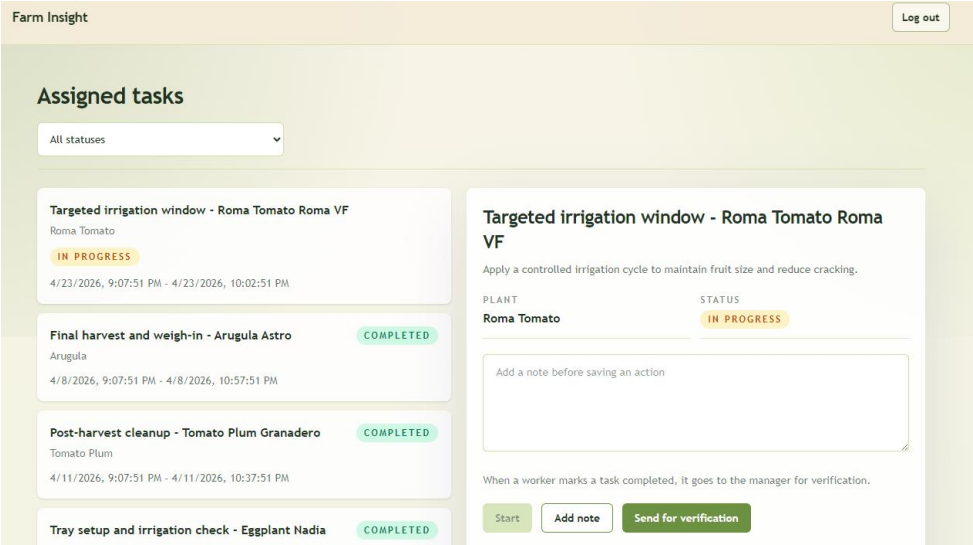


6.4 Worker Interface Page

The worker interface is separated from the manager interface. Workers do not have access to the overview, finance, inventory, task management, or plant management pages. Instead, they use a simplified page that focuses only on the information they need to perform their assigned work.

6.4.1 Workers Page

The Workers page is designed for users with worker-level access. It allows workers to view their assigned tasks and relevant work details. This page keeps the interface simple by showing only the necessary information, without giving access to manager-only system sections.



7. Maintenance, Evolution, and Reflection

7.1 Maintenance Plan

The Farm Management System is built around closely connected modules for plants, tasks, finance, inventory, and reporting, so the maintenance plan should focus on practical issues that are likely to appear in those daily workflows.

Maintenance Type	What it means in this system	Example
Corrective	Fixing faults or incorrect behavior in existing features	Fix a bug where a cancelled task can still be marked as completed by a worker
Adaptive	Changing the system to fit a new environment or technical requirement	Adjust authentication or session handling for a new hosting platform or an updated PostgreSQL version
Perfective	Improving usability, performance, structure, or maintainability without changing the core purpose	Improve task filters, dashboard summaries, and inventory low-stock alerts

These examples fit the current system because task status handling, login access, reporting, and inventory monitoring are all part of the supported workflows and modules already defined in the project.

7.2 Evolution Roadmap for v2.0

The current version covers the core farm workflow, but a later version should extend the system in areas that improve coordination, decision-making, and real field usage. Features such as recurring task support, reporting, and browser-based access already suggest natural directions for growth.

Proposed v2.0 Feature	Why it matters	Link to Lehman's Laws
Notifications and reminders for recurring tasks	Improves worker coordination and helps recurring work get completed on time	Continuing Change / Continuing Growth
Advanced reporting and trend analytics	Users will need better decision support as more task, harvest, finance, and inventory data is collected	Continuing Growth / Declining Quality unless improved
Mobile-friendly worker task view	The system should better support workers using it in real farm conditions	Continuing Change / Increasing Complexity

As software evolves, user expectations usually increase and the system becomes more complex. Because of that, the Farm Management System should not remain static after the first release. It should be improved deliberately so that growth in features does not reduce clarity, consistency, or usability.

7.3 Reflection Note

If I were rebuilding this project now, I would choose an iterative and incremental development process model. This would suit the project better than trying to finalize everything early, because the system changed several times while the requirements, diagrams, and structure became clearer. The project includes multiple connected areas such as plants, tasks, harvests, finance, sales, inventory, and reporting, so building and validating them in smaller steps would have been a better fit than trying to define everything in one pass.

This process model makes sense because much of the improvement in the project came through refinement. The behavioral modelling was corrected to keep only supported actors, improve use case naming, and make the diagrams more consistent with the real system scope. That shows that feedback led to meaningful corrections, which is exactly the kind of situation where iterative work is stronger than a rigid one-shot approach.

Several things went wrong during the project. First, the scope was initially a bit too broad, because the system includes many linked domains and it was easy to start thinking about advanced features before the core workflows were fully stabilized. Second, some diagrams included or risked including unsupported actors, which made the modelling less accurate at first. Third, naming was not always fully consistent across use cases, behavioral diagrams, and structural models. Fourth, some structure decisions were made before the requirements were

fully settled, which made later revisions necessary. These were not fatal problems, but they did create extra rework.

If I were doing the project again, I would change a few things. I would lock the scope earlier and clearly separate core features from later ideas from the beginning. I would define user roles, entity names, and naming conventions much earlier so every diagram and section uses exactly the same language. I would also review each model against the real implemented system before submission, especially checking whether actors, modules, and relationships match the database-backed design. Finally, I would keep a changelog from the start and work in smaller validated steps so that each correction is easier to track and apply consistently across the whole report.

Overall, the project improved a lot through revision. The final result is stronger because the requirements, behavioral models, architecture, and database structure were gradually aligned more closely with each other. That experience makes iterative and incremental development the process model I would choose now.

8. Tools and Technologies Used

8.1 Frontend Technologies

The frontend of the Farm Management System is built using React and Tailwind CSS. React was chosen because it supports a component-based structure, which is suitable for dividing the interface into reusable parts such as dashboards, plant pages, task lists, finance tables, and inventory views. Tailwind CSS was used to style the interface in a simple and consistent way, making it easier to build clean layouts, responsive pages, and reusable UI elements without writing large amounts of custom CSS.

8.2 Backend Technology

The backend of the system is developed using Django. Django was selected because it provides a strong structure for handling business logic, routing, authentication, form processing, and communication with the database. It is well suited for a system like this one because it helps organize features such as plant management, task handling, finance records, inventory updates, and reporting within one clear backend framework.

8.3 Database Technology

The system uses PostgreSQL as its database management system. PostgreSQL was chosen because it is reliable, supports relational data very well, and works effectively with structured schemas that include many linked tables and constraints. This is important for the Farm Management System, since the project depends heavily on relationships between users, farms, plots, greenhouses, plants, tasks, harvest records, finance records, sales data, and inventory records.

Layer	Technology
Frontend	React, Tailwind CSS
Backend	Django
Database	PostgreSQL

8. Conclusion

This final report consolidates all coursework stages into one corrected and internally consistent design for the Farm Management System. It brings together the refined requirements, process and behavioral models, database structure, architectural and structural design, and the final maintenance and evolution planning, supported throughout by the relevant diagrams and aligned system documentation.